SCAPE-ZK: Scalable Privacy-Preserving EHR Sharing with Proof-Carrying Aggregate Authorization and Context-Bound Delegation

Varit Mesri, Pankul Pankosol, Yanisa Kandamrongrak
Sirindhorn International Institute of Technology, Thammasat University, Thailand
6622771317@g.siit.tu.ac.th, 6622771523@g.siit.tu.ac.th, 6622790176@g.siit.tu.ac.th

***Abstract*—Secure and scalable sharing of Electronic Health Records (EHRs) across multiple healthcare domains remains a critical challenge due to stringent privacy requirements, inefficient authorization mechanisms, and limited support for dynamic delegation. Existing blockchain-based access control frameworks often suffer from attribute leakage, per-request verification overhead, and weak integration across authentication, authorization, and data transformation layers.**

In this paper, we propose SCAPE-ZK, a scalable privacy-preserving EHR sharing framework with proof-carrying aggregate authorization and context-bound delegation. SCAPE-ZK adopts a Cloud–Edge–Blockchain architecture and introduces a unified, end-to-end secure workflow. It employs a client-centric authentication model in which Data Users (DUs) locally generate zk-SNARK proofs from self-sovereign identity credentials, ensuring strict data minimization and attribute privacy. To achieve scalable authorization, SCAPE-ZK integrates zero-knowledge proofs with BLS aggregate signatures to enable proof-carrying aggregate authentication, reducing on-chain verification complexity from $\mathcal{O}(n)$ to $\mathcal{O}(1)$.

To support secure cross-domain data sharing, SCAPE-ZK incorporates a context-bound proxy re-encryption (PRE) mechanism that binds delegation to identity, data, policy, and authorization epoch, preventing replay and misuse while enabling efficient revocation without re-encrypting EHR data. In addition, SCAPE-ZK introduces a verification-first retrieval paradigm, where integrity (via Merkle proofs), authorization consistency, and transformation correctness are validated prior to decryption, ensuring strong end-to-end security guarantees. Experimental results demonstrate that SCAPE-ZK significantly improves scalability and reduces on-chain verification overhead compared to existing approaches, making it practical for large-scale, cross-domain healthcare environments.

***Index Terms*—Access Control, Aggregate Authentication, Blockchain, CP-ABE, Electronic Health Records, Fog Computing, Proxy Re-Encryption, Self-Sovereign Identity, Zero-Knowledge Proofs.**

I. Introduction

The secure and seamless exchange of Electronic Health Records (EHRs) across multiple healthcare domains is essential for modern collaborative healthcare, enabling accurate diagnosis, continuity of care, and data-driven medical research [1], [2]. However, EHRs contain highly sensitive personal and clinical information, making them prime targets for unauthorized access, inference attacks, and data misuse. Traditional centralized Identity and Access Management (IAM) systems suffer from single points of failure, poor interoperability, and vulnerability to large-scale breaches, resulting in fragmented data silos and limited cross-institutional collaboration [3], [4].

Blockchain has emerged as a promising decentralized infrastructure for access control, auditability, and trust management in healthcare systems [6], [7]. Systems such as MedRec [1] and subsequent works [2], [3], [5] demonstrate tamper-evident logging and removal of centralized trust. However, due to privacy and scalability constraints, EHRs cannot be stored directly on-chain. Hybrid architectures therefore store encrypted data off-chain (e.g., cloud or IPFS) while anchoring integrity proofs on-chain [11], [12]. While scalable, these designs introduce new challenges in ensuring secure, efficient, and flexible cross-domain access control.

To enhance privacy, recent approaches adopt Self-Sovereign Identity (SSI), Verifiable Credentials (VCs), and Zero-Knowledge Proofs (ZKPs) to enable attribute-based authentication without disclosure [8], [9]. Aggregate signature techniques further improve efficiency in multi-user scenarios [10]. However, existing frameworks such as XAuth [6], SSL-XIoMT [8], and VC aggregation systems [9] primarily focus on authentication in isolation, without addressing end-to-end system integration, scalability, and secure delegation.

Despite these advances, several key limitations remain.

First, existing systems often rely on *external or semi-trusted entities* to process authentication logic, which may expose sensitive credential information or expand the attack surface. Even when privacy-preserving techniques are used, the lack of strict client-side control weakens data minimization guarantees.

Second, most blockchain-based IAM frameworks adopt *per-request verification*, resulting in linear complexity $\mathcal{O}(n)$ and high gas costs under large-scale concurrent access [15]. This significantly limits scalability in real-world deployments.

Third, current Proxy Re-Encryption (PRE) mechanisms are typically *static and weakly bound to context*, making them vulnerable to misuse such as replay, cross-record substitution, and stale delegation [12]. Efficient and secure revocation remains particularly challenging.

Fourth, existing architectures lack *end-to-end consistency guarantees*. Authentication, authorization, storage, and transformation are treated as disjoint processes, leaving gaps between verified identity, delegated access, and retrieved data integrity [16]. As noted in recent surveys [17], achieving a

unified, scalable, and privacy-preserving design remains an open problem.

To address these challenges, we propose **SCAPE-ZK**, a decentralized privacy-preserving access control framework for collaborative EHR sharing. The framework adopts a *Cloud–Edge–Blockchain architecture* and establishes a unified, end-to-end secure workflow that jointly addresses privacy, scalability, and cross-domain interoperability.

SCAPE-ZK enforces a *client-centric privacy model*, where DUs locally generate zk-SNARK proofs from SSI/VC credentials, ensuring that sensitive attributes never leave the user device. To improve efficiency under repeated access, we introduce a *two-tier Context-Bound Policy Compliance Proof (CPCP)* mechanism. Specifically, the design consists of: (i) a *session-level proof* that validates credential authenticity and policy compliance, and (ii) a *request-level proof* that binds each access request to the verified session context. By separating these responsibilities, expensive cryptographic validation is performed only once per session, while subsequent requests incur only lightweight proof generation, significantly reducing amortized proving cost without weakening privacy guarantees.

To achieve scalable authorization, SCAPE-ZK integrates a *proof-carrying aggregate authentication* mechanism, in which Fog nodes batch validated requests and produce a single BLS aggregate signature. This enables constant-cost on-chain verification, reducing complexity from $\mathcal{O}(n)$ to $\mathcal{O}(1)$ [9], [10], [15].

For secure cross-domain sharing, SCAPE-ZK employs a *context-bound proxy re-encryption (PRE)* scheme that tightly binds delegation to identity, data, policy, and authorization epoch. This prevents replay and cross-context misuse while enabling efficient revocation without re-encrypting EHR data [12]. Furthermore, a *verification-first retrieval* workflow ensures that integrity, authorization consistency, and transformation correctness are validated prior to decryption, guaranteeing that only fresh and authorized ciphertexts are accessed.

The main contributions of this work are summarized as follows:

- **Client-side privacy-preserving authentication with two-tier CPCP.** We design a zk-SNARK-based authentication mechanism in which each DU locally proves possession of a valid verifiable credential and satisfaction of an access policy T without revealing underlying attributes [8], [9]. The proposed two-tier CPCP structure comprises a session-level proof for credential validation and policy compliance, and a lightweight request-level proof for context binding. This separation amortizes the cost of expensive zero-knowledge proof generation across multiple requests while preserving strong privacy guarantees, including attribute hiding, commitment consistency, and context integrity.
- **Proof-carrying aggregate authorization.** We construct a proof-carrying authorization mechanism by integrating zk-proofs with BLS aggregate signatures, where each request is bound to a context digest and its corresponding proof. Fog nodes batch validated tuples (ctx_i, π_i) and produce a single aggregate signature, enabling the blockchain to verify multiple requests using a constant number of pairing operations. This reduces verification complexity from $\mathcal{O}(n)$ to $\mathcal{O}(1)$ and significantly improves scalability in high-concurrency environments.
- **Context-bound proxy re-encryption with efficient revocation.** We design a dynamic PRE mechanism in which re-encryption keys are derived as $RK_{DO\to DU}^{(\mathsf{ctx},ver)} = \mathsf{ReKeyGen}(SK_{DO}, PK_{DU}, H(\mathsf{ctx} \,\|\, ver))$, binding delegation to a cryptographic context and authorization epoch. The proxy transforms only the encrypted symmetric key component, leaving the EHR ciphertext unchanged. This prevents replay, cross-record substitution, and stale delegation, while enabling revocation through lightweight epoch updates without data re-encryption.
- **Verification-first secure retrieval with optimized integrity validation.** We introduce a verification-first retrieval workflow in which the DU validates storage integrity through compact Merkle verification, authorization consistency under (ctx, ver), and PRE transformation correctness before decryption. By separating storage integrity from delegation correctness and using compact-leaf commitments, the design reduces retrieval-side verification overhead while preserving freshness, correctness, and consistency.

Overall, the proposed framework provides a secure, scalable, and privacy-preserving solution for cross-domain EHR sharing, bridging the gap between advanced cryptographic techniques and practical deployment requirements in modern healthcare systems.

II. RELATED WORK

A. Blockchain-Based Healthcare Data Sharing and IAM

Blockchain technology has been widely adopted to address the inherent vulnerabilities of centralized electronic health records (EHRs) and Identity and Access Management (IAM) systems.

Early foundational works, such as MedRec [1], demonstrated the feasibility of using blockchain for medical data access and permission management. Building on this, Han et al. [2] and Nekkalapudi et al. [4] proposed hybrid and decentralized methods for patient care to improve healthcare outcomes through secure data exchange. To specifically address the interoperability among medical consortiums, Xue et al. [3] and Ajayi et al. [5] introduced cross-domain authentication schemes and inter-healthcare exchange architectures.

While blockchain provides immutability, storing raw EHRs on-chain poses severe privacy and storage issues. To mitigate this, Liu et al. (BPDS) [11] proposed an architecture where original medical records are stored securely in the cloud, while only the data indexes are reserved in a tamper-proof blockchain. Recent studies in 2024, such as those by Eltabakh et al. [20] and Husnain et al. [21], further reinforced this by

proposing decentralized frameworks to mitigate data silos in EHR sharing.

Furthermore, addressing privacy in identity storage, Friebe et al. [13] introduced DecentID, a decentralized privacy-preserving identity system. Despite these advancements, many of these systems still struggle with the single-point-of-failure in traditional Single Sign-On (SSO) frameworks [14] and incur high computational overhead when handling fine-grained access policies. To address this, our framework introduces a *lightweight hybrid encryption with Merkle-based integrity anchoring*, utilizing decentralized IPFS for storage while anchoring only the constant-size Merkle root on-chain to strictly guarantee data immutability without burdening the network.

B. Privacy-Preserving Cross-Domain Authentication

To achieve secure authentication across isolated domains without compromising user privacy, various cryptographic protocols have been integrated into blockchain IAM. Chen et al. [6] proposed XAuth, a privacy-preserving cross-domain authentication scheme that utilizes a Multiple Merkle Hash Tree to ensure efficient verification.

Recently, the integration of Self-Sovereign Identity (SSI) and Zero-Knowledge Proofs (ZKP) has gained significant attention to guarantee data minimization. Hak and Fugkeaw [8] introduced SSL-XIoMT, leveraging SSI and ZKP to facilitate anonymous cross-domain data sharing. This trend is supported by Cocco and Tonelli [22], who proposed an SSI-blockchain model for digital healthcare transformation.

However, a major paradigm shift is required when applying these advanced protocols in real-world clinical settings. Deploying heavy cryptographic primitives directly on resource-constrained devices, such as mobile phones used by medical staff, remains a critical bottleneck. Recent studies in 2025 strongly corroborate this limitation; for instance, Rajasekaran et al. [18] highlighted the severe computational overhead of zk-SNARKs in Cloud-IoT healthcare sensors, while Perešíni et al. [19] emphasized the massive memory and resource consumption when executing zk-SNARKs on mobile clients.

Consequently, emerging research has shifted toward fog-assisted architectures. Reshi and Sholla [23] and recent studies on fog-assisted IoT clouds [24] explored offloading cryptographic tasks to edge nodes. Building upon this paradigm, our proposed system offers a *Fog-assisted policy-aware authentication with computation offloading*. By delegating heavy zk-SNARKs proof generation to local Fog nodes, we enable fast, mobile-friendly identity verification without leaking sensitive attributes.

C. Scalable Architectures and Secure Data Delegation

To overcome the scalability limitations of traditional monolithic blockchains, sharding technology and Edge/Fog computing have been widely introduced. Sorsoontornnirat et al. [15] proposed DYNASSO, a context-aware IAM framework that enables secure Single Sign-On (SSO) over sharded blockchains. Similarly, Thirasak et al. [12] introduced SSX-EHRs, a secure cross-domain sharing model that leverages dynamic proxy re-encryption (PRE) to facilitate scalable data transformation.

Beyond on-chain scalability, handling the authentication of multiple entities efficiently is crucial. While Zhou et al. [10] and Ahmed et al. [9] proposed multiparty authentication and VC aggregation schemes, verifying massive concurrent requests on-chain still incurs linear $\mathcal{O}(n)$ gas costs. Recent works in 2024 and 2025 have explored aggregate signatures and ZKP for large-scale systems, such as privacy-preserving AI [25] and federated learning [26], yet the on-chain verification bottleneck persists. Our framework overcomes this by achieving *aggregate authentication with constant-cost* $\mathcal{O}(1)$ *on-chain verification*, enabling the Smart Contract to authenticate batched BLS signatures via a single pairing operation.

Furthermore, managing access revocation in PRE remains resource-intensive. Recent attribute-based PRE schemes [28] and cross-domain consortium models [29], as well as version-controlled CP-ABE mechanisms [27], attempt to address this but often require extensive ciphertext updates. In contrast, our system introduces a *context-bound and epoch-based proxy re-encryption*. This novel technique binds the re-encryption key to a specific authorization version (epoch), allowing instant access revocation without the need to re-encrypt the massive underlying EHR data.

Ultimately, our comprehensive framework bridges the existing gaps by synergizing Fog-assisted aggregate authentication, sharded consortium blockchain, and epoch-based PRE to ensure secure, low-latency, and highly scalable EHR sharing.

III. Proposed System

A. SCAPE-ZK System Model

We consider a decentralized, privacy-preserving cross-domain EHR sharing system built upon a *Cloud–Edge–Blockchain architecture*. The system integrates Self-Sovereign Identity (SSI), Zero-Knowledge Proofs (ZKP), Ciphertext-Policy Attribute-Based Encryption (CP-ABE), and Proxy Re-Encryption (PRE) to enable secure, scalable, and patient-centric data access across multiple healthcare domains.

Fig. 1 illustrates the overall architecture, and Table II highlights the functional advantages of our framework compared to existing schemes.

The system follows a decentralized Cloud–Edge–Blockchain architecture with strict separation between identity ownership, cryptographic execution, and verification. We define the participating entities as follows:

- **Data Owners (DOs):** Patients who generate and own Electronic Health Records (EHRs). Each DO defines a fine-grained access policy T over their data and encrypts EHRs under a hybrid encryption mechanism before outsourcing. DOs also participate in generating delegation tokens for cross-domain sharing.

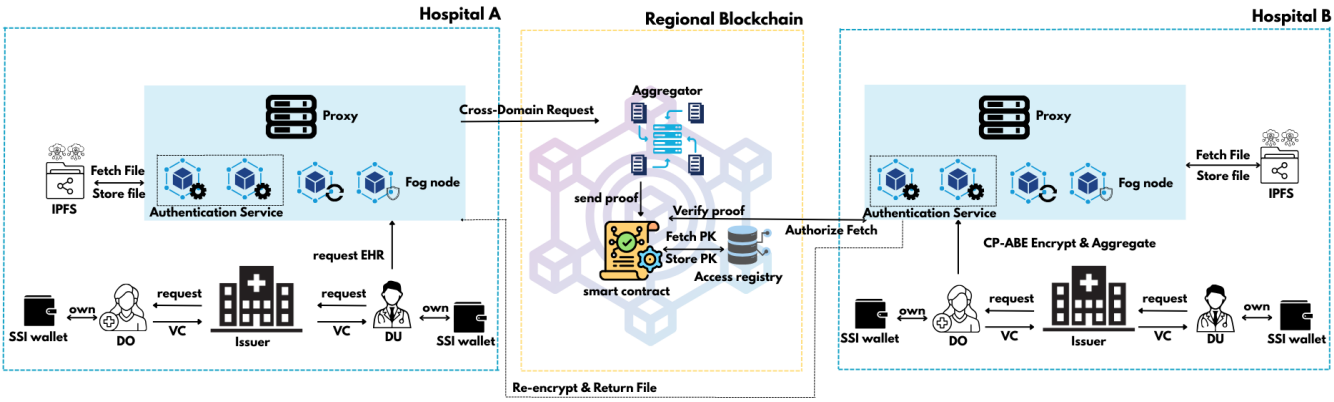

Fig. 1. System architecture of the proposed framework.

- **Data Users (DUs):** Authorized entities (e.g., doctors or healthcare providers) who request access to EHRs. Each DU holds a set of certified attributes embedded in a verifiable credential and locally generates privacy-preserving authentication proofs without revealing attribute values.
- **Issuer (Healthcare Authority):** A trusted authority responsible for identity verification and issuing *Verifiable Credentials (VCs)* under the Self-Sovereign Identity (SSI) model. The issuer signs user attributes but does not participate in authentication or data access thereafter.
- **SSI Wallet:** A user-controlled secure module that stores Decentralized Identifiers (DIDs), cryptographic key pairs, and issued credentials. All sensitive attribute information and proof witnesses remain strictly within the wallet, ensuring that no external entity (including Fog nodes) learns private attributes.
- **Fog Nodes (Edge Gateways):** Semi-trusted computational nodes deployed at the edge. Unlike conventional designs, Fog nodes *do not learn user attributes or witness information*. Their responsibilities are limited to:
  - *Proof Verifier and Aggregator:* Verify zero-knowledge proofs generated by DUs and aggregate valid authentication tuples.
  - *Signature Aggregator:* Produce BLS aggregate signatures over request-proof bindings.
  - *Proxy Re-Encryption Executor:* Perform context-bound ciphertext transformation without accessing plaintext or decryption keys.
- **Consortium Blockchain:** A permissioned distributed ledger that maintains global consistency and trust. It provides:
  - Smart contracts for enforcing access control and authorization policies,
  - On-chain verification of aggregated signatures and zero-knowledge proofs,
  - Immutable logging of access events, authorization states, and integrity anchors.
- **Decentralized Storage (IPFS):** A distributed storage system used to store encrypted EHRs. Only lightweight metadata (e.g., content identifiers CID and Merkle root H_{root}) are anchored on-chain, ensuring scalability and tamper-evident storage.

a) Context Binding.: To ensure end-to-end security across all phases, every access request and cryptographic transformation is bound to a context:

$$\mathsf{ctx} = (DID_{\text{DU}}, CID, H(T), \tau, ver),$$

where τ denotes freshness and ver denotes the authorization epoch. This context is consistently embedded into authentication proofs, signatures, and re-encryption operations, preventing replay, misuse, and cross-context attacks.

B. Threat Model

We consider a realistic adversarial setting in decentralized healthcare systems, where multiple entities may behave maliciously, attempt to infer sensitive information, or collude across system layers.

1) Adversary Capabilities: We assume the following adversarial capabilities:

- **External Adversary:** Can eavesdrop on communication channels, replay messages, inject forged requests, and attempt man-in-the-middle attacks.
- **Honest-but-Curious Storage Nodes (IPFS):** Follow protocol execution but attempt to infer sensitive information from stored ciphertexts and metadata.
- **Semi-Trusted Fog Nodes:** Execute assigned protocols correctly but attempt to learn user identities, attributes, or plaintext data. However, Fog nodes do not have access to secret keys or proof witnesses.
- **Malicious Data Users:** Attempt to gain unauthorized access by forging credentials, reusing stale proofs, or exploiting delegation mechanisms.
- **Colluding Adversaries:** Multiple entities (e.g., Fog node and malicious DU) may collude to infer hidden attributes, recover symmetric keys, or bypass access policies.

2) Leakage Model: The system reveals only minimal and well-defined leakage:

$$\mathcal{L} = \{|T|, |CT|, \text{access pattern}, \text{batch size}\},$$

while hiding all sensitive attributes, plaintext data, and secret keys. No entity learns the underlying attribute values or policy satisfaction beyond a single-bit authorization outcome.

3) Security Assumptions: The security of the system relies on the following assumptions:

- The symmetric encryption scheme is IND-CPA (or AEAD-secure for integrity).
- The CP-ABE scheme is IND-CPA secure (optionally with policy-hiding).
- The zk-SNARK system satisfies completeness, computational soundness, and zero-knowledge.
- The BLS signature scheme is EUF-CMA secure in the random oracle model.
- The proxy re-encryption (PRE) scheme is unidirectional, non-interactive, and secure against key recovery and ciphertext manipulation.
- Hash functions $H(\cdot)$ are collision-resistant and modeled as random oracles when required.
- The Merkle tree construction is collision-resistant.

TABLE I
NOTATION USED IN THE PROPOSED SYSTEM

Symbol	Description
M	Plaintext Electronic Health Record (EHR)
k	Symmetric encryption key
CT_M	Symmetric ciphertext of M
CT_k	CP-ABE encryption of key k
CT_{EHR}	Encrypted EHR package $\{CT_M, CT_k\}$
T	Access policy (LSSS or tree structure)
PK_{Issuer}	Public key of the Issuer
$PK_{\text{DU}}, SK_{\text{DU}}$	DU key pair
$PK_{\text{Fog}}, SK_{\text{Fog}}$	Fog node key pair
VC	Verifiable Credential
DID	Decentralized Identifier
$H(\cdot)$	Cryptographic hash function
h_i	Hash of ciphertext $CT_{\text{EHR},i}$
H_{root}	Merkle root hash
CID	Content identifier (IPFS)
ctx	Context binding tuple $(DID_{\text{DU}}, CID, H(T), \tau, ver)$
G_1, G_2, G_T	Bilinear groups of order p
$e(\cdot,\cdot)$	Bilinear pairing function

Phase 1: System Initialization and Context-Bound Secure EHR Storage

Phase 1 establishes the cryptographic and architectural foundation of the proposed framework, including decentralized identity bootstrapping, credential issuance, context-bound hybrid encryption of EHRs, integrity-preserving batch commitment, and scalable off-chain storage. Unlike a conventional encrypt-then-upload design, the proposed phase explicitly binds each encrypted record to its policy digest and storage context at creation time, thereby strengthening end-to-end consistency across subsequent authentication, delegation, and retrieval phases.

Step 1: Decentralized Identity Registration and Credential Issuance: Each participant, including a Data Owner (DO) and a Data User (DU), initializes a self-sovereign identity under the SSI model.

- The user generates a public/private key pair:

$$(PK_u, SK_u) \leftarrow \mathsf{KeyGen}(1^\lambda).$$

- A decentralized identifier is derived as:

$$DID_u = \mathsf{GenDID}(PK_u).$$

- The corresponding DID document is formed as

$$DDO_u = \{DID_u, PK_u, \text{metadata}\},$$

  and anchored on the consortium blockchain.
- The user submits a credential request to the Issuer, who verifies the real-world identity and certified role/attribute information.
- Upon successful verification, the Issuer generates a verifiable credential

$$VC_u = \mathsf{Sign}_{SK_{\text{Issuer}}}(\text{attr}_u, DID_u, t_{\mathsf{exp}}),$$

  where attr_u denotes the certified attribute set and t_{exp} denotes the credential expiration time.

The issued credential is stored locally in the user's SSI wallet together with the corresponding private key. This design avoids centralized identity repositories and ensures that certified attributes remain under user control until selectively proven in later phases.

Step 2: Context-Bound Hybrid Encryption of EHR Data: Let M denote an EHR generated by a DO, and let T denote the fine-grained access policy specified by the DO. To protect large medical files efficiently while enabling policy-enforced access, the system adopts a hybrid encryption mechanism with explicit context binding.

- The DO first computes a policy digest

$$\psi = H(T),$$

  which serves as a compact and immutable representation of the intended access rule.
- The DO samples fresh randomness $r \xleftarrow{\$} \{0,1\}^\lambda$ and derives a one-time symmetric key

$$k = \mathsf{KDF}(r \,\|\, DID_{\text{DO}} \,\|\, \psi).$$

  By deriving k from fresh randomness and the policy digest, the resulting encryption is cryptographically bound to the originating owner and policy context.
- The plaintext EHR is encrypted using a symmetric authenticated encryption scheme:

$$CT_M = \mathsf{Enc}_{\mathsf{AE}}(k, M; \mathsf{aad}),$$

  where the associated data is defined as

$$\mathsf{aad} = (DID_{\text{DO}}, \psi, t_{\text{up}}).$$

  This ensures that ciphertext integrity is additionally tied to the owner identity, policy digest, and upload timestamp.
- The symmetric key is then protected under the CP-ABE public parameters PP_{ABE} using access policy T:

$$CT_k = \mathsf{Enc}_{\text{CP-ABE}}(\mathsf{PP}_{\text{ABE}}, k, T).$$

If desired, this component can be instantiated using a policy-hiding CP-ABE construction so that the detailed policy structure is not directly exposed in the ciphertext.

- To strengthen cross-phase consistency, the DO computes a ciphertext binding tag

$$\mathsf{tag} = H(CT_M \,\|\, CT_k \,\|\, \psi \,\|\, DID_{\text{DO}}).$$

- The final encrypted EHR package is formed as

$$CT_{\text{EHR}} = \{CT_M, CT_k, \psi, \mathsf{tag}\}.$$

This construction provides three important properties. First, large EHR payloads are protected efficiently using symmetric encryption. Second, access control is cryptographically embedded into CT_k through CP-ABE rather than enforced by the storage provider. Third, the inclusion of ψ and tag creates an explicit cryptographic linkage between the encrypted payload, its governing policy, and the originating owner, which later supports authentication consistency and secure delegation.

Step 3: Fog-Assisted Batch Packaging and Merkle Commitment Construction: After encryption, the DO transmits encrypted records to its local Fog node for storage preparation. The Fog node assists only in aggregation, batching, and commitment construction; it does not learn plaintext data or the symmetric key.

- The DO uploads a batch of encrypted records

$$\{CT_{\text{EHR},1}, \ldots, CT_{\text{EHR},n}\}$$

to a nearby Fog node through a secure channel.

- For each encrypted record, the Fog node constructs a record descriptor

$$\mathsf{meta}_i = \big(\mathsf{tag}_i, \psi_i, DID_{\text{DO}}, t_i\big),$$

and computes the corresponding leaf commitment

$$h_i = H(CT_{\text{EHR},i} \,\|\, \mathsf{meta}_i).$$

- These leaf commitments are organized into a Merkle Hash Tree (MHT), where each parent node is recursively computed as

$$H_{\text{parent}} = H(h_{\text{left}} \,\|\, h_{\text{right}}).$$

- The final batch root is then obtained as

$$H_{\text{root}} = \mathsf{MerkleRoot}(\{h_i\}_{i=1}^{n}).$$

Compared with directly hashing raw ciphertext only, the proposed Merkle commitment binds each encrypted object together with its policy digest and record metadata. Consequently, any attempt to substitute ciphertexts, alter policy associations, or replay stale storage objects produces a mismatch at the Merkle verification stage. This improves structural integrity and makes the commitment semantically stronger than a simple content hash.

Step 4: Decentralized Storage and On-Chain Integrity Anchoring: To ensure availability and scalability, encrypted EHRs are stored off-chain in decentralized storage, while only compact integrity metadata are anchored on-chain.

- The Fog node uploads each encrypted package $CT_{\text{EHR},i}$, or an authenticated batch thereof, to IPFS and obtains the corresponding content identifier:

$$CID_i = \mathsf{IPFS.Store}(CT_{\text{EHR},i}).$$

- For each stored object, the system defines a compact storage record

$$\mathsf{Record}_i = \{CID_i, \mathsf{tag}_i, \psi_i, t_i\}.$$

- The batch anchor committed to the smart contract is then defined as

$$\mathsf{Anchor} = \{H_{\text{root}}, \mathsf{BatchID}, t_{\text{batch}}\},$$

where H_{root} commits to all encrypted records in the batch.

- The blockchain records the anchor by executing

$$\mathsf{Commit}(\mathsf{Anchor}).$$

This design separates bulky encrypted data from verifiable control metadata. IPFS provides scalable decentralized storage for ciphertext objects, while the blockchain stores only constant-size anchors that certify batch integrity and existence. Since each leaf hash incorporates both ciphertext and metadata, any off-chain tampering, substitution, or inconsistent record replay is efficiently detectable by recomputing the Merkle path against the anchored root.

Step 5: Phase Output: At the end of Phase 1, the system outputs the following artifacts:

$$\big(CT_{\text{EHR}},\ CID,\ \psi,\ \mathsf{tag},\ H_{\text{root}},\ VC\big).$$

These values serve as the cryptographic foundation for the subsequent phases. In particular, ψ and tag are later reused to bind authentication requests, enforce context-consistent delegation, and validate retrieved ciphertexts under the same policy and storage context.

Phase 2: Context-Bound Privacy-Preserving Authentication with Two-Tier CPCP

After securely storing encrypted EHRs in Phase 1, a Data User (DU) initiates a cross-domain access request. This phase enables privacy-preserving authorization through a *Context-Bound Policy Compliance Proof (CPCP)*. To improve efficiency in repeated-access scenarios, SCAPE-ZK adopts a *two-tier proof architecture*, which separates expensive credential validation from lightweight per-request authorization.

Let the DU possess a verifiable credential VC_{DU} issued by a trusted authority, containing certified attributes:

$$\mathcal{A}_{DU} = \{a_1, a_2, \ldots, a_m\}.$$

Each encrypted EHR is associated with a policy digest $\psi = H(T)$ (from Phase 1), and access is granted if:

$$\mathcal{A}_{DU} \models T.$$

Tier 1: Session-Level CPCP (Amortized Proof): This stage is executed infrequently (e.g., once per session or authorization epoch) and establishes the DU's eligibility under policy ψ without revealing attributes.

Step 1: Context Initialization: The DU defines a global authentication context:

$$\mathsf{ctx}_{\text{sess}} = (DID_{DU}, \psi, t_{\text{exp}}),$$

where t_{exp} is the credential validity period.

Step 2: Credential Commitment: To preserve privacy, the DU computes:

$$C_{VC} = \mathsf{Com}(VC_{DU}; r),$$

where r is randomness.

Step 3: Session CPCP Statement: The DU constructs the public statement:

$$x_{\text{sess}} = (C_{VC}, \psi, PK_{\text{Issuer}}, t_{\text{exp}}),$$

and witness:

$$w_{\text{sess}} = (\mathcal{A}_{DU}, \sigma_{VC}, r).$$

Define relation $\mathcal{R}_{\text{sess}}$ such that $\mathcal{R}_{\text{sess}}(x_{\text{sess}}, w_{\text{sess}}) = 1$ if:

$$\mathsf{Verify}_{PK_{\text{Issuer}}}(VC_{DU}, \sigma_{VC}) = 1, \tag{1}$$

$$\mathcal{A}_{DU} \models T \text{ where } H(T) = \psi, \tag{2}$$

$$\mathsf{ComOpen}(C_{VC}; VC_{DU}, r) = 1, \tag{3}$$

$$\tau_{\text{now}} \leq t_{\text{exp}}. \tag{4}$$

Step 4: Session Proof Generation: The DU generates:

$$\pi_{\text{sess}} \leftarrow \mathsf{Prove}(PK_{zk}, x_{\text{sess}}, w_{\text{sess}}).$$

This proof encapsulates credential validity and policy compliance and is reused across multiple requests.

Step 5: Authorization Token Derivation: Upon successful verification, a session-bound token is derived:

$$\mathsf{tok} = H(DID_{DU} \,\|\, C_{VC} \,\|\, \psi \,\|\, ver \,\|\, t_{\text{exp}}),$$

where ver denotes the current authorization epoch.

This token binds the DU identity, credential commitment, and policy context.

Tier 2: Request-Level CPCP (Lightweight Proof): For each access request, the DU generates only a lightweight proof that binds the request to the validated session.

Step 6: Context-Bound Access Request: The DU constructs:

$$\mathsf{Req}_i = (CID, \psi, \mathsf{tag}, \mathsf{nonce}_i, \tau_i),$$

and defines:

$$\mathsf{ctx}_i = (CID, \psi, \mathsf{tag}, \tau_i).$$

Step 7: Context Digest: To reduce circuit complexity, the request is compressed:

$$\mu_i = H(\mathsf{tok} \,\|\, \mathsf{ctx}_i \,\|\, \mathsf{nonce}_i).$$

Step 8: Lightweight CPCP Statement: The DU constructs:

$$x_i = (\mu_i, ver), \quad w_i = (\mathsf{tok}),$$

Define relation $\mathcal{R}_{\text{req}}$ such that $\mathcal{R}_{\text{req}}(x_i, w_i) = 1$ if:

$$\mathsf{tok} = H(DID_{DU} \,\|\, C_{VC} \,\|\, \psi \,\|\, ver \,\|\, t_{\text{exp}}), \tag{5}$$

$$\mu_i = H(\mathsf{tok} \,\|\, \mathsf{ctx}_i \,\|\, \mathsf{nonce}_i). \tag{6}$$

This proof avoids re-evaluating credential validity or policy satisfaction.

Step 9: Lightweight Proof Generation:

$$\pi_{\text{req},i} \leftarrow \mathsf{Prove}(PK_{zk}, x_i, w_i).$$

The resulting circuit is significantly smaller:

$$|\mathcal{C}_{\text{req}}| \ll |\mathcal{C}_{\text{sess}}|.$$

Step 10: Submission to Fog Node: The DU transmits:

$$(\mathsf{Req}_i, \pi_{\text{req},i}, \mathsf{tok})$$

to the Fog node.

The Fog node verifies:

$$\mathsf{Verify}(VK_{zk}, x_i, \pi_{\text{req},i}) \stackrel{?}{=} 1.$$

Step 11: Context-Bound Signature Generation: For valid requests, the Fog node computes:

$$m_i = H(\mu_i \,\|\, \pi_{\text{req},i}), \quad Q_i = H_1(m_i),$$

and generates:

$$\sigma_i = Q_i^{sk_{\text{Fog}}}.$$

Step 12: Authentication Tuple Construction: The final authentication tuple is:

$$(\mathsf{Req}_i, \pi_{\text{req},i}, \sigma_i),$$

which is forwarded for aggregation and batch verification in Phase 3.

Phase 3: Proof-Carrying Aggregate Authentication and Batch Verification

After generating context-bound authentication tuples in Phase 2, the system performs scalable verification using a *Proof-Carrying Authentication Batch (PCAB)* mechanism. This phase combines aggregate signature verification with zero-knowledge proof validation under a unified batch-processing pipeline, enabling high-throughput cross-domain authentication with strong security guarantees.

Let $\{(\mathsf{Req}_i, \pi_{zk,i}, \sigma_i)\}_{i=1}^n$ denote a batch of authentication tuples generated by Fog nodes, where each tuple is already bound to a context ctx_i and message digest $m_i = H(\mathsf{ctx}_i \,\|\, \pi_{zk,i})$.

Step 1: Batch Formation with Context Consistency: Each Fog node maintains a queue:

$$\mathcal{Q} = \{(\mathsf{Req}_1, \pi_{zk,1}, \sigma_1), \ldots, (\mathsf{Req}_n, \pi_{zk,n}, \sigma_n)\}.$$

Before aggregation, the Fog enforces *context consistency checks*:

- freshness of τ_i,
- uniqueness of nonce_i,
- structural validity of ctx_i.

Only valid tuples are admitted into the batch $\mathcal{B}$.

Step 2: Rogue-Key Resistant Aggregate Signature Construction: Each individual signature is:

$$\sigma_i = H_1(m_i)^{sk_{\text{Fog}}}.$$

To prevent rogue-key attacks and cross-batch forgery, the system applies *domain separation* and *batch binding*:

$$Q_i = H_1(\mathsf{domain} \,\|\, m_i \,\|\, i),$$

and constructs the aggregate signature:

$$\sigma_{\text{agg}} = \prod_{i=1}^{n} \sigma_i, \quad Q_{\text{agg}} = \prod_{i=1}^{n} Q_i.$$

Additionally, the Fog computes a batch identifier:

$$\mathsf{BID} = H(\{m_i\}_{i=1}^{n}),$$

which binds all requests into a single batch context.

The resulting payload is:

$$\mathsf{Payload} = \big(\{\mathsf{Req}_i, \pi_{zk,i}, m_i\}_{i=1}^{n}, \mathsf{BID}, \sigma_{\text{agg}}\big).$$

Step 3: On-Chain Constant-Cost Aggregate Verification: The smart contract verifies the batch using bilinear pairing:

$$e(\sigma_{\text{agg}}, g) \stackrel{?}{=} e(Q_{\text{agg}}, PK_{\text{Fog}}),$$

where $PK_{\text{Fog}} = g^{sk_{\text{Fog}}}$.

By bilinearity:

$$e(\sigma_{\text{agg}}, g) = e\left(\prod_{i=1}^{n} Q_i^{sk_{\text{Fog}}}, g\right) \tag{7}$$

$$= e\left(\prod_{i=1}^{n} Q_i, g^{sk_{\text{Fog}}}\right) \tag{8}$$

$$= e(Q_{\text{agg}}, PK_{\text{Fog}}). \tag{9}$$

This achieves:

$$\mathcal{O}(1)$$

pairing cost for verifying all signatures.

a) Security Note.: The inclusion of (domain, i) in Q_i ensures non-malleability and prevents adversarial reordering or substitution within aggregated batches.

Step 4: Two-Level Verification Pipeline (Early Filtering): To optimize verification cost, the contract adopts a two-level pipeline:

b) Level 1: Aggregate Authentication Filter:

- Verify aggregate signature
- Reject entire batch if invalid

c) Level 2: Proof-Carrying Authorization Verification: For each i:

$$\mathsf{Verify}(VK_{zk}, x_i, \pi_{zk,i}) \stackrel{?}{=} 1.$$

Each proof guarantees:

- valid credential,
- policy satisfaction,
- binding to $(CID, \psi, \mathsf{tag})$,
- freshness and non-replay.

Invalid proofs are selectively discarded, while valid requests proceed.

d) Optimization Insight.: This design ensures that expensive ZKP verification is performed only after cryptographic authenticity is globally validated, significantly reducing worst-case computation under adversarial flooding.

Step 5: Authorization Commitment and Event Anchoring: For each valid request, the contract records:

$$\mathsf{Auth}_i = H(\mathsf{ctx}_i \,\|\, m_i \,\|\, \mathsf{BID}).$$

The system logs:

$$\mathsf{Log}(\mathsf{Auth}_i, \mathsf{approved}), \quad \mathsf{Log}(\mathsf{Auth}_i, \mathsf{rejected}).$$

Optionally, these logs can be organized into a Merkle tree:

$$H_{\text{auth-root}} = \mathsf{MerkleRoot}(\{\mathsf{Auth}_i\}),$$

which enables efficient auditing and cross-phase verification.

Phase 4: Context-Bound Verifiable Proxy Re-Encryption for Cross-Domain Delegation

After successful authentication and authorization in Phase 3, the system enables secure and verifiable delegation of decryption capability across domains. This phase introduces a *Context-Bound Verifiable Proxy Re-Encryption (CB-VPRE)* mechanism, which transforms ciphertexts without exposing plaintext data and without requiring the Data Owner (DO) to be online.

Let an approved request be associated with:

$$(\mathsf{ctx}_i, m_i, \mathsf{Auth}_i),$$

and the stored encrypted record be:

$$CT_{\text{EHR}} = \{CT_M, CT_k, \psi, \mathsf{tag}\}.$$

Step 1: Authorization Consistency and Context Binding: The Fog node retrieves the approved authorization context from the blockchain:

$$(\mathsf{ctx}_i, \mathsf{Auth}_i) \in \mathsf{AuthRegistry}.$$

The context is defined as:

$$\mathsf{ctx}_i = (DID_{DU}, CID, \psi, \mathsf{tag}, \tau_i, ver_i).$$

The Fog verifies:

- Auth_i is valid and not revoked,
- ver_i matches the current system epoch,
- $(CID, \psi, \mathsf{tag})$ matches stored ciphertext metadata.

This ensures that transformation is strictly tied to an authenticated and non-revoked request.

Step 2: Context-Bound Re-Encryption Key Derivation: Instead of a generic PRE key, the system derives a *context-bound re-encryption key*:

$$RK^{(i)}_{DO\to DU} = \mathsf{ReKeyGen}(SK_{DO}, PK_{DU}, H(\mathsf{ctx}_i)).$$

This ensures:

- delegation is valid only for this request,
- reuse across different ciphertexts or contexts is impossible,
- replay across domains is prevented.

e) Security Insight.: Even if $RK^{(i)}_{DO\to DU}$ is exposed, it cannot be reused due to strong binding to ctx_i.

Step 3: Verifiable Key Transformation: The Fog node applies proxy re-encryption only to the key component:

$$CT_k^{DU} = \mathsf{PRE.Transform}(CT_k, RK^{(i)}_{DO\to DU}, \mathsf{ctx}_i).$$

To ensure correctness and prevent malicious transformation, the Fog generates a transformation proof:

$$\pi_{\text{pre},i} \leftarrow \mathsf{Prove}_{\text{PRE}}(CT_k, CT_k^{DU}, \mathsf{ctx}_i).$$

This proof certifies that:

- CT_k^{DU} is correctly derived from CT_k,
- the transformation uses a valid re-encryption key,
- the operation is bound to ctx_i.

f) Remark.: The proof can be instantiated using a lightweight NIZK or pairing-based consistency proof depending on the PRE construction.

Step 4: Ciphertext Reconstruction and Binding Preservation: The transformed ciphertext is:

$$CT^{DU}_{\text{EHR}} = \{CT_M, CT_k^{DU}, \psi, \mathsf{tag}\}.$$

Importantly:

- CT_M remains unchanged,
- (ψ, tag) preserve integrity and policy binding from Phase 1,
- transformation affects only the access layer.

This design ensures:

$$\text{Confidentiality} \perp \text{Delegation},$$

i.e., data confidentiality is independent of delegation operations.

Step 5: Epoch-Based Revocation Enforcement: The system maintains a global epoch counter:

$$ver \in \mathbb{N}.$$

Upon revocation or policy update:

$$ver \leftarrow ver + 1.$$

A re-encryption key is valid only if:

$$ver_i = ver.$$

Thus:

- all previous $RK^{(i)}$ become invalid,
- no ciphertext re-encryption is required,
- revocation cost is constant-time.

g) Advantage.: This achieves *forward revocation security* without re-encrypting stored EHRs.

Step 6: Adaptive and Load-Aware Transformation Scheduling: To support large-scale deployments, the system employs adaptive Fog selection.

Let $\mathcal{F}$ be the set of Fog nodes. The optimal node is:

$$Fog^* = \arg\min_j \left(\alpha L_j + \beta D_j + \gamma Q_j\right),$$

where:

- L_j = computational load,
- D_j = network latency,
- Q_j = queue length.

The selected node executes:

$$(CT_k^{DU}, \pi_{\text{pre},i}) = \mathsf{PRE.Transform}_{Fog^*}(CT_k, RK^{(i)}, \mathsf{ctx}_i).$$

This prevents bottlenecks and ensures balanced workload distribution.

Step 7: Output of Delegation Phase: The final output delivered to the DU is:

$$\left(CT^{DU}_{\text{EHR}}, \pi_{\text{pre},i}, \mathsf{ctx}_i\right).$$

Phase 5: Verification-First Context-Consistent Data Retrieval and Decryption

After proxy re-encryption in Phase 4, the transformed ciphertext is delivered to the authorized Data User (DU). This phase ensures that data retrieval, validation, and decryption are performed in a strictly *verification-first* manner, preventing any unauthorized, tampered, or context-inconsistent ciphertext from being decrypted.

Unlike naive designs, the proposed phase explicitly separates: (i) *storage integrity verification* for the original encrypted record and (ii) *delegation correctness verification* for the transformed key component. This separation both improves correctness and reduces verification overhead.

Let the DU receive:

$$\left(CT^{DU}_{\text{EHR}}, \pi_{\text{pre},i}, \mathsf{ctx}_i\right),$$

where:

$$CT^{DU}_{\text{EHR}} = \{CT_M, CT_k^{DU}, \psi, \mathsf{tag}\}.$$

Step 1: Ciphertext Retrieval and Structural Validation: The DU retrieves the encrypted record from decentralized storage using the content identifier CID. Let the retrieved object correspond to the originally stored ciphertext package:

$$CT_{\text{EHR}} = \{CT_M, CT_k, \psi, \mathsf{tag}\}.$$

The DU first checks:

- correctness of the retrieved object with respect to CID (e.g., via content-addressable verification),
- structural validity of both CT_{EHR} and CT^{DU}_{EHR}.

Any malformed or inconsistent structure results in immediate rejection.

Step 2: Efficient Merkle-Based Storage Integrity Verification: To verify that the retrieved ciphertext has not been tampered with, the DU validates the Merkle proof against the on-chain root H_{root}.

Instead of hashing the entire ciphertext again, the verification is performed over a compact leaf commitment derived in Phase 1:

$$h_i = H(CID \,\|\, \mathsf{tag} \,\|\, \psi \,\|\, DID_{\text{DO}} \,\|\, t_i).$$

Let π_{Merkle} denote the authentication path. The DU verifies:

$$\mathsf{VerifyMerkle}(h_i, \pi_{\text{Merkle}}, H_{\text{root}}) \stackrel{?}{=} 1.$$

This design ensures:

- **Integrity**: the stored ciphertext and metadata are unchanged,
- **Efficiency**: avoids re-hashing large EHR payloads,
- **Consistency**: aligns exactly with Phase 1 commitment.

Step 3: Context Consistency and Authorization Validation: The DU reconstructs:

$$\mathsf{ctx}_i = (DID_{DU}, CID, \psi, \mathsf{tag}, \tau_i, ver_i),$$

and verifies that:

$$(\mathsf{ctx}_i, \mathsf{Auth}_i) \in \mathsf{AuthRegistry}.$$

Additionally, freshness is enforced:

$$|\tau_{\text{now}} - \tau_i| \leq \Delta.$$

This step guarantees that the request:

- is authorized,
- has not been revoked,
- is not a replay.

Step 4: Verifiable Proxy Re-Encryption Validation: The DU verifies correctness of the delegated transformation:

$$\mathsf{Verify}_{\text{PRE}}(CT_k, CT_k^{DU}, \pi_{\text{pre},i}, \mathsf{ctx}_i) \stackrel{?}{=} 1.$$

This ensures:

- CT_k^{DU} is correctly derived from CT_k,
- the transformation is bound to ctx_i,
- no malicious modification occurred at the Fog node.

Step 5: Context-Bound Symmetric Key Recovery: Upon successful verification, the DU recovers the symmetric key:

$$k = \mathsf{Dec}(SK_{DU}, CT_k^{DU}; \mathsf{ctx}_i).$$

Due to context binding, decryption succeeds only if:

- identity,
- ciphertext,
- authorization context

are all consistent.

Step 6: Authenticated Data Decryption: The DU decrypts:

$$M = \mathsf{Dec}_{\mathsf{AE}}(k, CT_M; \mathsf{aad}),$$

where:

$$\mathsf{aad} = (DID_{\text{DO}}, \psi, t_{\text{up}}).$$

This guarantees:

- confidentiality of M,
- integrity of ciphertext,
- binding to original encryption context.

Step 7: Optional Lightweight Transformation Consistency Check (Optimized): To avoid incorrect comparisons with the original ciphertext tag, the system optionally verifies a lightweight transformation-binding digest:

$$\mathsf{tag}_i^{\text{pre}} = H(CT_k^{DU} \,\|\, \mathsf{ctx}_i \,\|\, \mathsf{tag}),$$

and checks:

$$\mathsf{tag}_i^{\text{pre}} \stackrel{?}{=} H(CT_k^{DU} \,\|\, \mathsf{ctx}_i \,\|\, \mathsf{tag}).$$

This step:

- preserves end-to-end binding across transformation,
- avoids re-hashing large ciphertext,
- replaces the incorrect comparison between CT_k and CT_k^{DU}.

IV. Security Analysis

In this section, we analyze the security of the proposed framework under the adversarial model defined in Section III. The revised construction combines: (i) SSI/VC-based identity bootstrapping, (ii) a context-bound policy-compliance proof generated locally by the DU, (iii) proof-carrying aggregate authentication for scalable authorization, (iv) context-bound verifiable proxy re-encryption for secure delegation, and (v) verification-first context-consistent decryption for end-to-end data access. Our analysis focuses on correctness, confidentiality, attribute privacy, authentication unforgeability, authorization soundness, integrity, replay resistance, revocation soundness, and secure delegation.

A. Security Assumptions and Adversarial Model

Let λ be the security parameter. We assume the following standard security properties for the underlying primitives.

- The authenticated encryption scheme $\mathsf{AE} = (\mathsf{Enc}_{\mathsf{AE}}, \mathsf{Dec}_{\mathsf{AE}})$ is IND-CPA secure and provides ciphertext integrity under associated data.
- The CP-ABE scheme is IND-CPA secure under the public parameters PP_{ABE}. If a policy-hiding variant is instantiated, then the access policy is hidden beyond its allowed leakage.
- The commitment scheme Com is computationally binding and hiding.
- The zk-SNARK system $(\mathsf{Setup}, \mathsf{Prove}, \mathsf{Verify})$ satisfies completeness, computational soundness, and zero-knowledge.

- The BLS signature scheme is EUF-CMA secure in the random oracle model under the hardness assumption of the underlying bilinear group.
- The hash functions H and H_1 are collision resistant; H_1 is additionally modeled as a hash-to-curve random oracle where required.
- The Merkle-tree hash is collision resistant.
- The proxy re-encryption primitive is unidirectional, non-interactive, ciphertext-private, and key-private, and its transformation correctness is publicly verifiable through $\mathsf{Verify}_{\mathrm{PRE}}$.
- The smart contract logic and on-chain state transition rules execute faithfully once deployed.

We consider a probabilistic polynomial-time adversary $\mathcal{A}$ that may eavesdrop on communication channels, inject, delay, replay, or reorder messages, observe blockchain contents, compromise IPFS storage nodes, and behave as an honest-but-curious Fog node. The adversary may also corrupt malicious DUs and attempt collusion between a Fog node and a DU. However, $\mathcal{A}$ cannot break the above primitives except with negligible probability. The Issuer is trusted for correct identity verification and credential issuance.

B. Security Goals

The proposed framework aims to achieve the following properties.

1) **EHR confidentiality:** no unauthorized entity can learn the plaintext EHR.
2) **Attribute privacy:** no verifier, Fog node, or blockchain observer learns the DU's sensitive attributes beyond the fact that the authorization statement is valid.
3) **Authentication unforgeability:** no adversary can forge a fresh valid request-proof-signature tuple accepted by the contract.
4) **Authorization soundness:** only DUs holding valid, non-expired credentials whose certified attributes satisfy the target policy can obtain approval.
5) **Integrity and consistency:** tampering with stored ciphertexts, ciphertext metadata, transformation outputs, or authorization state is detectable.
6) **Replay resistance and revocation soundness:** stale requests, stale transformed ciphertexts, or pre-revocation delegation artifacts cannot be reused successfully.
7) **Secure delegation:** Fog nodes can transform only the delegated key component without learning the plaintext or usable decryption keys.
8) **End-to-end context consistency:** the retrieved ciphertext, authorization context, and transformed key component remain bound across Phases 1–5.

C. Leakage Profile

The framework reveals only limited public leakage necessary for functionality. Let

$$\mathcal{L} = \{|CT|,\ |T| \text{ or } H(T),\ \text{batch size, access pattern, timing metadata}\}.$$

If policy-hiding CP-ABE is used, the leakage can be reduced to a policy digest or other minimal public encoding. No plaintext EHR, credential content, DU attribute values, witness values, symmetric key, or DO/DU secret key is revealed.

D. Correctness of the Protocol

We first establish functional correctness under honest execution.

Theorem 1 (Protocol Correctness)**.** *Assume that: (i) the Issuer issues a valid credential VC_{DU}, (ii) the DU's certified attributes satisfy the policy T, (iii) the DU generates a valid zero-knowledge proof $\pi_{zk,i}$ for the public statement x_i, (iv) the Fog node correctly signs the bound message digest and includes the request in a valid batch, (v) the smart contract accepts the authorization entry, and (vi) the PRE transformation is performed correctly for the same context ctx_i. Then the authorized DU correctly recovers the symmetric key k and decrypts the EHR plaintext M.*

Proof. By correctness of the commitment scheme, the committed credential opens properly. By zk-SNARK completeness, a valid witness yields an accepting proof, so $\mathsf{Verify}(VK_{zk}, x_i, \pi_{zk,i}) = 1$. By correctness of BLS aggregation, the aggregate signature equation holds for honestly signed request digests. Hence the authorization entry associated with ctx_i is accepted on-chain. By correctness of the PRE transformation, the transformed key ciphertext CT_k^{DU} decrypts under SK_{DU} and the same context ctx_i to recover the original symmetric key k. Finally, by correctness of the authenticated encryption scheme, $\mathsf{Dec}_{\mathsf{AE}}(k, CT_M; \mathsf{aad})$ returns M. Therefore the DU obtains the correct plaintext. □

E. Confidentiality of Outsourced EHR Data

The stored EHR package has the form

$$CT_{\mathrm{EHR}} = \{CT_M, CT_k, \psi, \mathsf{tag}\},$$

where

$$CT_M = \mathsf{Enc}_{\mathsf{AE}}(k, M; \mathsf{aad}), \qquad CT_k = \mathsf{Enc}_{\text{CP-ABE}}(\mathsf{PP}_{\mathrm{ABE}}, k, T),$$

and $\psi = H(T)$.

Theorem 2 (EHR Confidentiality)**.** *If the authenticated encryption scheme is IND-CPA secure and the CP-ABE scheme is IND-CPA secure, then any PPT adversary that does not possess a valid decryption capability for the policy T has only negligible advantage in distinguishing the outsourced ciphertext of one chosen plaintext EHR from that of another equal-length plaintext.*

Proof. We use a standard hybrid argument.

Hybrid H_0. This is the real system, where

$$CT_M = \mathsf{Enc}_{\mathsf{AE}}(k, M_b; \mathsf{aad}), \qquad CT_k = \mathsf{Enc}_{\text{CP-ABE}}(\mathsf{PP}_{\mathrm{ABE}}, k, T),$$

for challenge bit $b \in \{0, 1\}$.

Hybrid H_1. Replace the encapsulated symmetric key with an independent random key $k^\star$:

$$CT_k^\star = \mathsf{Enc}_{\text{CP-ABE}}(\mathsf{PP}_{\mathrm{ABE}}, k^\star, T).$$

Any distinguisher between H_0 and H_1 yields an IND-CPA adversary against the CP-ABE scheme. Hence

$$|\Pr[\mathcal{A} \Rightarrow 1 \mid H_0] - \Pr[\mathcal{A} \Rightarrow 1 \mid H_1]| \leq \mathrm{negl}(\lambda).$$

Hybrid H_2**.** Keep $CT_k^\star$ and replace CT_M with an authenticated encryption of M_{1-b} under $k^\star$. Any distinguisher between H_1 and H_2 yields an IND-CPA adversary against AE. Therefore

$$|\Pr[\mathcal{A} \Rightarrow 1 \mid H_1] - \Pr[\mathcal{A} \Rightarrow 1 \mid H_2]| \leq \mathrm{negl}(\lambda).$$

In H_2, the resulting ciphertext is independent of b, except for negligible terms. By triangle inequality, the adversary's distinguishing advantage in the real system is negligible. □

F. Privacy of DU Attributes and Anonymous Authentication

In the revised Phase 2, the DU generates the proof locally. The witness remains entirely inside the DU-side SSI wallet and is never revealed to the Fog node. The public statement contains only the commitment to the credential, policy digest, request context, issuer public key, and expiration metadata.

Theorem 3 (Attribute Privacy)**.** *Under the hiding property of* Com *and the zero-knowledge property of the zk-SNARK, any PPT adversary observing the request transcript, Fog-side verification process, blockchain logs, aggregate authentication payloads, and transformed ciphertexts learns no more than the validity of the authorization statement and the allowed public leakage.*

Proof. The adversary observes $(x_i, \pi_{zk,i}, m_i, \sigma_i)$, where x_i contains only public metadata and the commitment $C_{VC} = \mathsf{Com}(VC_{DU}; r)$. By hiding of the commitment scheme, C_{VC} computationally hides the committed credential and the underlying attribute set. By zero-knowledge of the zk-SNARK, there exists a simulator $\mathcal{S}$ that, given only the public statement x_i, produces a simulated proof $\widetilde{\pi}_{zk,i}$ such that

$$(x_i, \widetilde{\pi}_{zk,i}) \approx_c (x_i, \pi_{zk,i}).$$

Hence the proof leaks nothing beyond the validity of the encoded statement. Since the Fog signs only the digest $m_i = H(\mathsf{ctx}_i \| \pi_{zk,i})$, and since ctx_i contains no secret witness component, the signature leaks no additional attribute information. Therefore the adversary learns no hidden attribute values beyond the permitted public leakage. □

G. Authorization Soundness

Theorem 4 (Authorization Soundness)**.** *If the zk-SNARK is sound and the Issuer's credential signature scheme is EUF-CMA secure, then no PPT adversary can cause the system to approve a request for which the underlying credential is invalid, expired, not properly committed, or policy-incompatible, except with negligible probability.*

Proof. Suppose an adversary causes acceptance of a request whose hidden witness fails at least one required condition: the credential signature is invalid, the certified attribute set does not satisfy T, the commitment does not open correctly, or the credential/request freshness condition fails. For the request to be approved, the smart contract must eventually rely on an accepting proof $\pi_{zk,i}$ for public statement x_i. By definition of the NP relation encoded in the proof system, acceptance implies existence of a witness w satisfying all required checks. If no such witness exists, the adversary has violated soundness of the zk-SNARK. If the witness contains a forged valid credential signature, then the adversary breaks unforgeability of the Issuer's signature scheme. Thus false authorization can occur only with negligible probability. □

H. Unforgeability of Authentication Tuples

Each authenticated request tuple has the form

$$(\mathsf{Req}_i, \pi_{zk,i}, \sigma_i), \qquad m_i = H(\mathsf{ctx}_i \| \pi_{zk,i}), \qquad \sigma_i = Q_i^{sk_{\mathrm{Fog}}},$$

where $Q_i = H_1(\mathsf{domain} \| m_i \| i)$.

Theorem 5 (Authentication Tuple Unforgeability)**.** *Assuming BLS is EUF-CMA secure and H is collision resistant, no PPT adversary can forge a fresh valid tuple* $(\mathsf{Req}^\star, \pi^\star, \sigma^\star)$ *accepted by the batch verification procedure without either obtaining an honest Fog signature on the corresponding digest or breaking one of the underlying assumptions.*

Proof. Assume an adversary outputs a fresh tuple such that

$$e(\sigma^\star, g) = e(H_1(\mathsf{domain} \| m^\star \| i^\star), PK_{\mathrm{Fog}}),$$

where

$$m^\star = H(\mathsf{ctx}^\star \| \pi^\star).$$

If the message corresponding to $m^\star$ was never honestly signed, then $\sigma^\star$ is a fresh BLS forgery, contradicting EUF-CMA security. Otherwise, if the adversary maps a different pair $(\mathsf{ctx}^\star, \pi^\star)$ to a previously signed digest, then it produces a collision in H. Therefore successful forgery occurs only with negligible probability. □

I. Correctness and Soundness of Aggregate Authentication

The revised batch verification uses the proof-carrying aggregate authentication mechanism:

$$\sigma_{\mathrm{agg}} = \prod_{i=1}^{n} \sigma_i, \qquad Q_{\mathrm{agg}} = \prod_{i=1}^{n} Q_i,$$

with the on-chain check

$$e(\sigma_{\mathrm{agg}}, g) \stackrel{?}{=} e(Q_{\mathrm{agg}}, PK_{\mathrm{Fog}}).$$

Theorem 6 (Aggregate Signature Correctness)**.** *For honestly generated signatures under a registered Fog public key, the aggregate verification equation holds:*

$$e(\sigma_{\mathrm{agg}}, g) = e(Q_{\mathrm{agg}}, PK_{\mathrm{Fog}}).$$

Proof. For each i, $\sigma_i = Q_i^{sk_{\mathrm{Fog}}}$. Therefore

$$\sigma_{\mathrm{agg}} = \prod_{i=1}^{n} \sigma_i = \prod_{i=1}^{n} Q_i^{sk_{\mathrm{Fog}}}.$$

By bilinearity,

$$\begin{aligned} e(\sigma_{\text{agg}}, g) &= e\left(\prod_{i=1}^{n} Q_i^{sk_{\text{Fog}}}, g\right) \\ &= \prod_{i=1}^{n} e(Q_i^{sk_{\text{Fog}}}, g) \\ &= \prod_{i=1}^{n} e(Q_i, g^{sk_{\text{Fog}}}) \\ &= e\left(\prod_{i=1}^{n} Q_i, PK_{\text{Fog}}\right) = e(Q_{\text{agg}}, PK_{\text{Fog}}). \end{aligned}$$

Hence the equation is correct. □

Theorem 7 (Batch Authentication Soundness)**.** *Assuming the aggregate signature verification is secure, the Fog public key is registered correctly, and the zk-SNARK is sound, no adversary can cause a malformed batch containing forged or unauthorized requests to be accepted, except with negligible probability.*

Proof. If the aggregate pairing equation fails, the batch is rejected immediately. Thus suppose it passes. Then the aggregate signature must be consistent with the registered Fog public key unless the adversary breaks EUF-CMA security of the aggregate signing mechanism. After passing the aggregate filter, each proof $\pi_{zk,i}$ is verified individually against its public statement. Any unauthorized request in the batch would therefore require an accepting proof for a false statement, violating zk-SNARK soundness. Hence no malformed batch can yield unauthorized approvals except with negligible probability. □

J. Integrity of Outsourced EHRs and On-Chain Consistency

In the revised storage phase, each Merkle leaf commits not only to the ciphertext package but also to its associated metadata. Specifically,

$$h_i = H(CT_{\text{EHR},i} \| \mathsf{meta}_i),$$

where meta_i includes at least the binding tag, policy digest, owner information, and timestamp or equivalent record metadata.

Theorem 8 (Tamper Detection and Metadata Consistency)**.** *If the Merkle-tree hash is collision resistant, then any modification, substitution, or metadata-inconsistent replay of an outsourced ciphertext record is detected except with negligible probability.*

Proof. The on-chain root is derived recursively from the leaf hashes h_i. For a modified ciphertext or altered metadata tuple to verify under the same Merkle proof and same root, the adversary must either find a collision in the leaf hash

$$H(CT'_{\text{EHR},i} \| \mathsf{meta}'_i) = H(CT_{\text{EHR},i} \| \mathsf{meta}_i),$$

or find collisions in the internal Merkle nodes leading to the same root. Both events contradict collision resistance. Therefore any tampering or inconsistent substitution is detected with overwhelming probability. □

K. Security of Context-Bound Verifiable PRE

The revised delegation phase uses a context-bound re-encryption key

$$RK_{DO\to DU}^{(i)} = \mathsf{ReKeyGen}(SK_{DO}, PK_{DU}, H(\mathsf{ctx}_i)),$$

and produces a transformed ciphertext

$$CT_k^{DU} = \mathsf{PRE.Transform}(CT_k, RK_{DO\to DU}^{(i)}, \mathsf{ctx}_i),$$

together with a proof of correct transformation $\pi_{\text{pre},i}$.

Theorem 9 (Delegation Confidentiality)**.** *Assuming the PRE scheme is ciphertext-private and key-private, the CP-ABE ciphertext hides the session key, and the authenticated encryption is secure, then an honest-but-curious Fog node or external adversary observing $(CT_k, CT_k^{DU}, \pi_{\text{pre},i}, \mathsf{ctx}_i)$ cannot recover k or M except with negligible probability.*

Proof. The Fog receives only the protected key ciphertext CT_k, the context-bound re-encryption key, and public context information. By key privacy and ciphertext privacy of the PRE primitive, transformation does not reveal the underlying session key or usable decryption capability beyond the intended transformed ciphertext semantics. Since k remains hidden inside CT_k and the transformed output is only decryptable by the intended DU under the proper context, the Fog cannot recover k. Without k, the adversary cannot break the authenticated encryption of CT_M except with negligible probability. Thus both the symmetric key and plaintext remain confidential. □

Theorem 10 (Verifiable Transformation Soundness)**.** *If $\mathsf{Verify}_{\text{PRE}}$ is sound and the PRE scheme is correct, then no PPT adversary can produce a malformed transformed ciphertext CT_k^{DU} together with a proof $\pi_{\text{pre},i}$ that is accepted as a valid context-bound transformation of CT_k, except with negligible probability.*

Proof. Suppose an adversary outputs $(CT_k^{DU}, \pi_{\text{pre},i}, \mathsf{ctx}_i)$ such that

$$\mathsf{Verify}_{\text{PRE}}(CT_k, CT_k^{DU}, \pi_{\text{pre},i}, \mathsf{ctx}_i) = 1$$

but CT_k^{DU} is not a correct transformation of CT_k under the authorized context. Then the adversary has forged an accepting proof for an invalid transformation statement, contradicting soundness of the transformation-verification mechanism. Therefore any accepted transformed ciphertext must correspond to a valid context-bound transformation, except with negligible probability. □

Theorem 11 (Context-Bound Delegation Soundness)**.** *A transformed ciphertext CT_k^{DU} can be successfully decrypted only by the intended DU under the same approved authorization context ctx_i, except with negligible probability.*

Proof. The re-encryption key derivation binds the delegation to both the recipient public key PK_{DU} and the hash of the authorization context. Therefore any attempt to reuse the transformed ciphertext under a different user identity, different content identifier, different policy digest, different binding tag,

or different epoch results in a mismatch during either PRE verification, context checking, or final decryption. Successful misuse would therefore require breaking the context-binding property of the PRE construction, forging on-chain authorization state, or violating the correctness boundary of the decryption algorithm. Each event occurs only with negligible probability. □

L. Revocation Soundness

Theorem 12 (Epoch-Based Revocation Soundness)**.** *If the authorization epoch is incremented whenever a DU is revoked or a policy state changes, then any re-encryption artifact generated under an earlier epoch cannot be reused successfully after revocation, except with negligible probability.*

Proof. Every approved context contains the epoch component ver_i, and the system accepts only requests whose context matches the current on-chain epoch. When revocation occurs, the contract updates the epoch value to a fresh version. Any previously generated re-encryption key or transformed ciphertext remains bound to the older value. Since Phase 5 requires context consistency with the current authorization record, old transformation outputs fail verification or decryption after the epoch changes. Therefore revocation is enforced without re-encrypting the EHR body. □

M. Replay Resistance

Theorem 13 (Replay Resistance)**.** *Assuming nonce uniqueness, timestamp freshness, collision resistance of H, sound registry checking, and correct epoch handling, replay of an old request tuple or old transformed ciphertext under a new session or authorization state succeeds only with negligible probability.*

Proof. In the revised authentication phase, the Fog signs

$$m_i = H(\mathsf{ctx}_i \| \pi_{zk,i}),$$

where ctx_i includes the DU identity, target content identifier, policy digest, ciphertext binding tag, timestamp, and epoch. Any replay under changed context changes ctx_i, and hence changes m_i, unless a collision in H is found. Reusing an old tuple with the same context but stale timestamp is rejected by freshness checking. Reusing an old transformed ciphertext under a later epoch fails the context-consistency and revocation checks. Therefore replay is possible only if the adversary breaks nonce uniqueness, timestamp validation, registry correctness, epoch handling, or collision resistance, each of which occurs only with negligible probability. □

N. Verification-First Decryption Security

The revised retrieval phase requires the DU to verify the Merkle proof, authorization consistency, transformation proof, and context binding before decrypting the ciphertext.

Theorem 14 (Verification-First Decryption Soundness)**.** *If the DU accepts the ciphertext in Phase 5 and outputs a plaintext M, then except with negligible probability, the accepted plaintext corresponds to a ciphertext that is authentic, integrity-preserving, context-consistent, and correctly transformed under an approved authorization state.*

Proof. Acceptance in Phase 5 requires successful verification of four independent conditions: the Merkle proof matches the on-chain root, the authorization context exists and is fresh, the PRE transformation proof is valid, and authenticated decryption under the recovered key succeeds on the associated data. A forged ciphertext that passes all four checks would therefore imply either a Merkle collision, a forged authorization state, a forged PRE verification proof, or a break of AE ciphertext integrity. Under the stated assumptions, each event occurs only with negligible probability. Hence any accepted plaintext corresponds to a valid authorized retrieval of the intended record. □

O. End-to-End Security

We now combine the above results.

Theorem 15 (End-to-End Security)**.** *Under the stated assumptions, the proposed framework provides end-to-end confidentiality of EHR contents, privacy-preserving authentication, authentication unforgeability, authorization soundness, metadata-aware tamper detection, secure context-bound delegation, transformation verifiability, replay resistance, revocation soundness, and verification-first decryption security against PPT adversaries, except with negligible probability in λ.*

Proof. Confidentiality of outsourced EHRs follows from Theorem 2. Attribute privacy follows from Theorem 3. Authorization soundness follows from Theorem 4. Authentication tuple unforgeability and batch soundness follow from Theorems 5–7. Integrity and metadata consistency follow from Theorem 8. Delegation confidentiality, verifiable transformation soundness, and context-bound delegation follow from Theorems 9–11. Revocation soundness follows from Theorem 12. Replay resistance follows from Theorem 13. Verification-first decryption soundness follows from Theorem 14. Since all protocol-visible artifacts reveal only the allowed leakage profile and every successful adversarial strategy would imply breaking at least one underlying primitive or trusted state invariant, the total adversarial advantage is bounded above by the sum of negligible terms. Therefore the overall advantage is negligible in λ. □

V. EVALUATION

In this section, we present a comparative analysis of our proposed framework against existing schemes in terms of functionality and computational cost. Our evaluation focuses on decentralized EHR sharing systems that employ advanced cryptographic techniques, including zero-knowledge proofs, aggregate signatures, and proxy re-encryption, to achieve secure and scalable cross-domain access control.

A. Functionality Comparison

We compare the functionality of **SCAPE-ZK** with representative schemes, namely XAuth [6], SSL-XIoMT [8], and Scheme [30]. These schemes are selected due to their relevance in decentralized authentication and secure cross-domain data sharing for IoT, IIoT, and healthcare environments.

Table II summarizes the comparison based on key functional properties required for practical and secure EHR sharing systems. The selected features capture essential aspects including privacy-preserving authentication, delegation capability, scalability, integrity assurance, and revocation support.

TABLE II
FUNCTIONAL COMPARISON OF SCAPE-ZK WITH EXISTING SCHEMES

Scheme	F1	F2	F3	F4	F5	F6
Scheme [6]	✓	×	✓	×	×	×
Scheme [8]	✓	×	✓	×	×	×
Scheme [30]	✓	×	✓	×	×	✓
SCAPE-ZK	✓	✓	✓	✓	✓	✓

Features: F1 = Privacy-Preserving Authentication
F2 = Efficient Delegation Mechanism
F3 = Blockchain-Based Access Control
F4 = Scalable Authorization
F5 = Verifiable Data Integrity
F6 = Efficient Revocation Support

As shown in Table II, existing schemes provide only partial support for the required functionalities in decentralized EHR sharing.

XAuth [6] and SSL-XIoMT [8] both support privacy-preserving authentication using zero-knowledge proofs and leverage blockchain for access control. However, they lack efficient delegation mechanisms (e.g., proxy re-encryption), scalable authorization (such as batch or aggregate verification), and verifiable data integrity. In addition, their revocation mechanisms are limited and do not support efficient updates in highly dynamic environments.

Scheme [30] adopts a self-sovereign identity (SSI) framework integrated with blockchain to achieve decentralized and privacy-preserving authentication. It supports efficient batch authentication through aggregate signatures and enhances credential management via a threshold-based revocation and tracking mechanism. However, its design primarily focuses on identity authentication rather than secure data sharing. Specifically, it does not support delegation mechanisms such as proxy re-encryption, lacks verifiable data integrity guarantees for outsourced data, and does not provide scalable authorization mechanisms beyond authentication-level batching.

In contrast, SCAPE-ZK is the only scheme that simultaneously supports all considered functionalities within a unified framework. It integrates privacy-preserving authentication with an efficient delegation mechanism, enables scalable authorization via proof aggregation, and ensures verifiable data integrity through cryptographic proofs. Furthermore, it provides an efficient revocation mechanism that avoids costly data re-encryption and supports dynamic updates.

TABLE III
NOTATION USED IN COMPUTATION COST ANALYSIS

Symbol	Description
T^{g}_{zk}	Zero-knowledge proof generation cost
T^{v}_{zk}	Zero-knowledge proof verification cost
$T^{\mathsf{sym}}_{\mathsf{enc}}$	Symmetric encryption cost
$T^{\mathsf{abe}}_{\mathsf{enc}}$	CP-ABE encryption cost
$T^{\mathsf{ecc}}_{\mathsf{enc}}$	ECC-based encryption / key exchange cost
T_{pair}	Bilinear pairing cost
T_{hash}	Cryptographic hash cost (general)
T^{ℓ}_{hash}	Lightweight hash cost (metadata-level hashing)
T_{pre}	Proxy re-encryption transformation cost
T^{v}_{pre}	Proxy re-encryption verification cost
T_{merk}	Standard Merkle proof verification cost
T^{*}_{merk}	Optimized Merkle verification cost (compact leaf using CID and metadata)
n	Number of concurrent access requests

Overall, SCAPE-ZK achieves a more comprehensive and balanced design by jointly addressing functionality, scalability, and verifiability, making it well-suited for large-scale, cross-domain EHR sharing environments.

B. Computation Cost Analysis

We evaluate the computational cost of SCAPE-ZK against XAuth [6], SSL-XIoMT [8], and Scheme [30], focusing on the online access workflow, including authentication, authorization, and secure data retrieval. The analysis decomposes costs into fundamental cryptographic operations and distinguishes between per-request overhead and system-level scalability.

Table IV compares the per-request computational cost across schemes, where the dominant overhead arises from either zero-knowledge proof operations or pairing-based verification, depending on the underlying design paradigm.

XAuth and SSL-XIoMT primarily rely on zero-knowledge proofs, and their computational cost is dominated by proof generation complexity, which scales with the size of the constraint system (e.g., identity or attribute circuits). As a result, both schemes incur consistent per-request overhead without benefiting from amortization.

In contrast, Scheme [30] adopts an SSI-based authentication model that combines zero-knowledge proofs with aggregate signature verification over bilinear pairings. Its verification cost consists of two components: (i) T^{v}_{zk} for proof verification, and (ii) pairing-based and group operations modeled as $\mathcal{O}(T_{\mathsf{pair}} + n \cdot T_{\mathsf{grp}})$. Although batch authentication reduces the number of pairing evaluations to a constant, the overall cost still grows linearly with the number of aggregated requests due to the required group exponentiations and scalar multiplications. Consequently, its amortized cost improves only partially and remains dependent on the batch size.

SCAPE-ZK differs fundamentally by introducing a proof amortization strategy at the authorization level. While it incurs higher initial proof generation cost due to richer policy constraints, this cost is shared across multiple requests within a session. As reflected in Table IV, the amortized proof cost decreases proportionally with n, making SCAPE-ZK significantly more efficient in repeated-access or session-based scenarios.

TABLE IV
PER-REQUEST COMPUTATION COST COMPARISON

Scheme	Proof Gen	Amortized Proof	Encrypt	Proof Ver	Integrity / Delegation
XAuth [6]	$T^g_{\mathsf{zk}}(\lvert\mathcal{C}_{\mathrm{id}}\rvert)$	$T^g_{\mathsf{zk}}(\lvert\mathcal{C}_{\mathrm{id}}\rvert)$	$--$	T^v_{zk}	$T_{\mathsf{merk}} + T_{\mathsf{hash}}$
SSL-XIoMT [8]	$T^g_{\mathsf{zk}}(\lvert\mathcal{C}_{\mathrm{attr}}\rvert)$	$T^g_{\mathsf{zk}}(\lvert\mathcal{C}_{\mathrm{attr}}\rvert)$	$T^{\mathsf{sym}}_{\mathsf{enc}} + T^{\mathsf{abe}}_{\mathsf{enc}} + T^{\mathsf{ecc}}_{\mathsf{enc}}$	T^v_{zk}	$T_{\mathsf{merk}} + T_{\mathsf{hash}}$
Scheme [30]	T^g_{zk}	$T^g_{\mathsf{zk}} + \frac{\mathcal{O}(T_{\mathsf{pair}} + nT_{\mathsf{grp}})}{n}$	$--$	$T^v_{\mathsf{zk}} + \mathcal{O}(T_{\mathsf{pair}} + nT_{\mathsf{grp}})$	T_{hash}
SCAPE-ZK	$T^g_{\mathsf{zk}}(\lvert C_{\mathsf{req}}\rvert)$	$\frac{T^g_{\mathsf{zk}}(\lvert C_{\mathsf{sess}}\rvert) + n \cdot T^g_{\mathsf{zk}}(\lvert C_{\mathsf{req}}\rvert)}{n}$	$T^{\mathsf{sym}}_{\mathsf{enc}} + T^{\mathsf{abe}}_{\mathsf{enc}}$	T^v_{zk}	$T^*_{\mathsf{merk}} + T^v_{\mathsf{pre}} + T^\ell_{\mathsf{hash}}$

TABLE V
AUTHORIZATION VERIFICATION SCALABILITY

Scheme	Off-chain Cost	On-chain Cost
XAuth [6]	$n \cdot T^v_{\mathsf{zk}}$	$n \cdot T_{\mathsf{hash}}$
SSL-XIoMT [8]	$n \cdot T^v_{\mathsf{zk}}$	$n \cdot T_{\mathsf{hash}}$
Scheme [30]	$T^v_{\mathsf{zk}} + \mathcal{O}(T_{\mathsf{pair}} + n \cdot T_{\mathsf{grp}})$	$n \cdot T_{\mathsf{hash}}$
SCAPE-ZK	$n \cdot T^v_{\mathsf{zk}}$	T_{pair}

From an encryption perspective, SCAPE-ZK employs a hybrid symmetric and CP-ABE design while eliminating ECC-based key exchange and avoiding pairing-based cryptographic operations during the online phase. This results in a more lightweight and practical execution model compared to SSL-XIoMT and Scheme [30].

While Table IV focuses on per-request overhead, Table V highlights system-level scalability. XAuth and SSL-XIoMT incur linear verification cost $n \cdot T^v_{\mathsf{zk}}$ due to independent verification of each request. Scheme [30] improves efficiency through batch authentication; however, its total verification cost remains linear due to aggregation overhead and group operations, limiting scalability under high concurrency.

In contrast, SCAPE-ZK enables proof-carrying aggregate authorization, allowing multiple requests to be verified collectively using a constant number of pairing operations. This reduces on-chain verification from $\mathcal{O}(n)$ to $\mathcal{O}(1)$, significantly improving scalability.

Overall, combining insights from both tables, SCAPE-ZK strategically shifts computational complexity toward client-side proof generation while achieving constant-cost verification at the system level. This design provides a balanced trade-off between computation and scalability, making it particularly suitable for large-scale, high-concurrency environments such as cross-domain EHR sharing.

REFERENCES

[1] A. Azaria, A. Ekblaw, T. Vieira, and A. Lippman, "MedRec: Using Blockchain for Medical Data Access and Permission Management," in *2016 2nd International Conference on Open and Big Data (OBD)*, 2016, pp. 25–30.

[2] G. Han, Y. Ma, Z. Zhang, and Y. Wang, "A hybrid blockchain-based solution for secure sharing of electronic medical record data," *PeerJ Computer Science*, vol. 11, e2653, 2025.

[3] L. Xue, H. Huang, F. Xiao, and W. Wang, "A Cross-Domain Authentication Scheme Based on Cooperative Blockchains Functioning With Revocation for Medical Consortiums," *IEEE Transactions on Network and Service Management*, vol. 19, no. 3, pp. 2409–2420, 2022.

[4] S. Nekkalapudi and R. K. Mishra, "Blockchain-Based Healthcare Outcome Improvement: A Decentralized Method of Patient Care," in *2024 First International Conference on Data, Computation and Communication (ICDCC)*, 2024.

[5] O. Ajayi, M. Abouali, and T. Saadawi, "Secured Inter-Healthcare Patient Health Records Exchange Architecture," in *2020 IEEE International Conference on Blockchain*, 2020.

[6] J. Chen, Z. Zhan, K. He, R. Du, D. Wang, and F. Liu, "XAuth: Efficient Privacy-Preserving Cross-Domain Authentication," *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 5, pp. 3301–3311, 2022.

[7] K. Dhampiban-udom, N. Koson, C. Udompongpipat, and S. Fugkeaw, "BZET-IAM: Enabling Blockchain-Based Zero Trust SSO for Dynamic Identity and Access Management," in *2025 22nd International Joint Conference on Computer Science and Software Engineering (JCSSE)*, 2025.

[8] L. Hak and S. Fugkeaw, "SSL-XIOMT: Secure, Scalable, and Lightweight Cross-Domain IoMT Sharing With SSI and ZKP Authentication," *IEEE Open Journal of the Computer Society*, 2025.

[9] I. Ahmed, K. Toyoda, T. Nakano, and T. H. Tran, "Efficient Verifiable Credential Aggregation With Blockchain Anchoring and zk-SNARKs," *IEEE Access*, vol. 13, 2025.

[10] Y. Zhou, L. Li, X. Zuo, and Y. Liu, "A Multiparty Authentication Scheme Based on Aggregate Certificateless Signature for Smart Healthcare," *IEEE Internet of Things Journal*, vol. 12, no. 17, pp. 15985–15998, 2025.

[11] J. Liu, X. Li, L. Ye, H. Zhang, X. Du, and M. Guizani, "BPDS: A Blockchain-based Privacy-Preserving Data Sharing for Electronic Medical Records," in *IEEE Global Communications Conference (GLOBECOM)*, 2017.

[12] K. Thirasak, D. Chainarong, T. Chuaphanngam, and S. Fugkeaw, "SSX-EHRs: secure and scalable cross-domain EHRs sharing with blockchain sharding and dynamic proxy re-encryption," *EURASIP Journal on Information Security*, vol. 2025, no. 15, 2025.

[13] S. Friebe, I. Sobik, and M. Zitterbart, "DecentID: Decentralized and Privacy-preserving Identity Storage System using Smart Contracts," in *2018 17th IEEE International Conference On Trust, Security And Privacy In Computing And Communications*, 2018.

[14] S. Roy and S. Matloob, "On Application of Blockchain to Enhance Single Sign-On (SSO) Systems," in *2021 IEEE 20th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*, 2021.

[15] T. Sorsoontornnirat, S. Imsaard, P. Topiya, and S. Fugkeaw, "DYNASSO: Enabling Secure and Dynamic SSO over Sharded Blockchains for Scalable and Context-Aware IAM," 2024.

[16] S. Fugkeaw, R. P. Gupta, and K. Worapaluk, "Secure and Fine-Grained Access Control With Optimized Revocation for Outsourced IoT EHRs With Adaptive Load-Sharing in Fog-Assisted Cloud Environment," *IEEE Access*, vol. 12, pp. 82753–82768, 2024.

[17] A. K. Singh, V. R. P. Kumar, G. Dehdasht, and F. P. Rahimian, "Comprehensive survey of blockchain consensus mechanisms: Analysis, applications, and future trends," *Computer Networks*, vol. 272, p. 111661, Sep. 2025.

[18] P. Rajasekaran, M. Duraipandian, and J. R. Albert, "Privacy-enhanced data compression using quantum zk-SNARKs and variational autoencoders in cloud-IoT based healthcare sensor data for medical applications," *AIP Advances*, vol. 15, no. 10, p. 105131, Oct. 2025.

[19] M. Perešíni, I. Homoliak, S. Olekšák, and S. Slávka, "FeatherWallet: A Lightweight Mobile Cryptocurrency Wallet Using zk-SNARKs," *arXiv preprint arXiv:2503.22717*, Mar. 2025.

[20] M. A. Eltabakh et al., "Blockchain-Enhanced Electronic Health Records: A Secure and Immutable Approach to EHR Management," *Journal of Engineering Research*, 2024.

[21] G. Husnain et al., "HealthChain: A blockchain-based framework for

secure and interoperable electronic health records (EHRs)," *IET Communications*, 2024.

[22] L. Cocco and R. Tonelli, "A Self-Sovereign Identity–Blockchain-Based Model Proposal for Deep Digital Transformation in the Healthcare Sector," *Future Internet*, vol. 16, no. 5, 2024.

[23] M. Reshi and S. Sholla, "Privacy enhancement technology for fog computing (PET-Fog) utilizing homomorphic encryption and zk-SNARKs," *Journal of ICT Standardization*, 2024.

[24] "Efficient and Secure Blockchain-Based Access Control for Fog-Assisted IoT Cloud in Electronic Medical Records Sharing," *Journal of Electrical Systems*, 2024.

[25] "Privacy-Preserving AI Collaboration on Blockchain Using Aggregate Signatures with Public Key Aggregation," *MDPI Applied Sciences*, 2024.

[26] "VerifBFL: Leveraging zk-SNARKs for A Verifiable Blockchained Federated Learning," *arXiv preprint*, 2025.

[27] G. Qiang et al., "VB-CP-ABE: An Improved CP-ABE Scheme Based on Blockchain Collaboration and Fine-Grained Version Control," *Journal of Computing and Electronic Information Management*, 2025.

[28] "An Attribute-Based Proxy Re-Encryption Scheme Supporting Revocable Access Control," *MDPI Electronics*, 2024.

[29] R. Xiong, J. Cheng, X. Dong, J. Pu and F. Shan, "Leveraging Consortium Blockchain for Secure Cross-Domain Data Sharing in Supply Chain Networks" in IEEE Transactions on Services Computing, vol. 18, no. 02, pp. 897-911, March-April 2025, doi: 10.1109/TSC.2025.3544130.

[30] Y. He et al., "Cross-Domain Identity Authentication Scheme for the IIoT Identification Resolution System Based on Self-Sovereign Identity" in IEEE Transactions on Networking, vol. 33, no. 06, pp. 2913-2928, Dec. 2025, doi: 10.1109/TON.2025.3578035.